



Online evaluation of the Kolmogorov–Smirnov test on arbitrarily large samples

Douglas O. Cardoso^{a,*}, Thalís D. Galeno^b

^a Smart Cities Research Center, Polytechnic Institute of Tomar, Tomar, Portugal

^b Department of Computer Engineering, Celso Suckow da Fonseca Federal Center for Technological Education, Petrópolis, RJ, Brazil

ARTICLE INFO

Dataset link: <https://github.com/moretoknow/driftwatch>, https://github.com/moretoknow/reedyks_experiments, <https://iew.technion.ac.il/serveng/callcenterdata/index.html>

Keywords:

Data streams
Online learning
Concept drift
Change detection

ABSTRACT

This paper presents an approximative online algorithm to perform the Kolmogorov–Smirnov test. There is a ubiquitous need for evaluating the fitness between statistical distributions and data samples, which this test conveniently meets. Taking some inspiration from the challenges of detecting concept drifts in data streams, our methodology shows how this goodness-of-fit statistical test can be used to detect such events, taking advantage of the fact that it is non-parametric and could be adapted to handle streams while keeping its original relatively small algorithmic complexity. The presented work focused on the one-sample test, which evaluates the hypothesis that a given univariate sample follows some reference distribution, for assessing an input stream with high precision in a time- and space-efficient fashion. The performance of our algorithm and some of the state-of-the-art methods were compared using synthetic and real data. We evaluated the accuracy, effectiveness and efficiency of these methods by making extensive experiments in multiple scenarios: varying reference distribution and its parameters, sample size, available memory, drift point and query interval. The results showed that our algorithm is advantageous in most cases, even with substantial restrictions of computational resources.

1. Introduction

The recent substantial increase of computing and electronic devices is one of the reasons why Big Data is a reality, and one of the biggest challenges it poses is data management. For instance, collecting data from Internet can be practically impossible without some kind of summarization. Detecting changes in the data distribution of an incrementally provided sample could be considered a primary task. Notwithstanding, it can be quite difficult due to the disposal of relevant information [1,2]. Change detection is an important issue in many contexts, as in data stream classification [3]. There are other examples: when analyzing the rate of packets in a high speed network [4], or the interval between signals of a certain type emitted when a meteor is detected by a telescope [5], a goal could be to identify periods in which the distribution of the frequency of key events is considered abnormal, and this could require to manage up to 1 TB of data per second [6].

In many applications it is impossible to keep the raw input, as the volume of data is a few orders of magnitude larger than the available memory. Furthermore, data streams are neither finite nor steady with respect to rate, i.e., the number of elements left in a time window cannot be known in advance [7]. To deal with such conditions, efficient

algorithms are needed, capable of operating in real time and with reasonable use of resources [8]. One way to solve this problem is to work on the empirical distribution of data. Considering this target, this paper presents a method to perform the Kolmogorov–Smirnov (KS) test on a given input stream. Among others, one interesting application of this method is to detect changes before a presumed distribution.

The KS test is a statistical method to assess whether a sample follows a certain distribution. Its usefulness is boosted by the fact that it is non-parametric: it can be applied assuming that the data obey any type of distribution. In a few words, the test is calculated by evaluating the density of subsets of the sorted input sample. This becomes harder when dealing with large samples which are incrementally provided. Therefore, an idealized asset is an algorithm whose processing has a constant cost for each observation, making the desired solution even more complex. Fortunately, a solution as this already exists [9], although it is not online. There are also works in the literature that enable the calculation of the test in an online fashion, but are more restricted with respect to processing granularity [10,11] or induced an error up to three orders of magnitude greater than the best we obtained in some experiments [12]. Thus, improving the estimation of the exact

* Corresponding author.

E-mail addresses: douglas.cardoso@ipt.pt (D.O. Cardoso), thalis.galeno@aluno.cefet-rj.br (T.D. Galeno).

URL: <https://docardoso.github.io/> (D.O. Cardoso).

results of the KS test under such efficiency constraints is still a major computational challenge.

This paper completes the proposal of an original algorithm to calculate the one-sample KS test online (i.e., considering an incrementally provided input sample and any reference distribution), concluding the maturation of preliminary results of the past [13]. The method is able, depending on the choice of an auxiliary data structure, to process each item in $O(\log m)$ time and obtain the value of the statistic evaluated in the KS test in $O(1)$: the parameter m controls the memory used, with no attachment to the sample size. Also according to such a data structure, our method can have a memory complexity of $O(m)$. In tests in varied scenarios, the algorithm showed results predominantly superior to other approaches for the same purpose, even when compared to the current state-of-the-art. Thus, it intends to fill the research gap between carrying out the KS test in the original exact but offline fashion and known online alternatives with a suboptimal performance regarding precision or efficiency.

The rest of the paper is organized as follows. In Section 2 we present an overview of the necessary concepts for understanding the proposed method. Some works directly related to the topic of this paper are indicated in Section 3. The proposed algorithm and some of its properties are described in Section 4. Tests and comparisons made with other methods are presented in Section 5. Finally, we indicate some concluding remarks and future works in Section 6.

2. Preliminaries

This section aims to present the basic concepts needed for the best understanding of the proposed method. The concept of population can be defined as the entire universe of items that are of interest to an experiment. A subset of this population is known as a sample. The sample space Ω is the set of possible outcomes for an experiment. A random variable X is a function that maps the result of an experiment to a single real value, i.e. $X : \Omega \rightarrow \mathbb{R}$. A distribution function, or Cumulative Distribution Function (CDF), is a function that defines the probability that an observation is less than or equal to a certain threshold, defined as $F(x) = \Pr[X \leq x]$.

Quantiles are used to summarize or characterize samples in a way more robust to outliers than the classical mean and variance options. They are a fundamental tool for statistical analysis. The quantile $Q(\phi)$ can be interpreted as the inverse function of the distribution function $F(x)$, i.e. $Q(F(x)) = x$. Another important concept is the Empirical Cumulative Distribution Function (ECDF) whose value at a certain point x , given a sample A , is defined as

$$F_A(x) = \frac{|\{a \in A : a \leq x\}|}{|A|}.$$

Evaluating whether there is a generative relationship between a sample with a reference distribution using the ECDF of the former and the CDF of the latter is the central idea of the KS test in its one-sample version. This test is based on the D statistic, informally referred to as the “KS distance”, which is capable of representing the degree of association between a reference distribution and a given sample. Its value is the length of the largest separation between CDF and ECDF: $D = \sup_x |F_A(x) - F(x)|$. For a sample $A = (X_1, X_2, \dots, X_n)$ in non-decreasing order (i.e., if $i \geq j$, then $X_i \geq X_j$) and a CDF $F(x)$, the KS distance is determined by the formulae

$$\begin{aligned} D^+ &= \max_i \left\{ \frac{i}{n} - F(X_i) \right\}, \\ D^- &= \max_i \left\{ F(X_i) - \frac{i-1}{n} \right\}, \text{ and} \\ D &= \max \{ D^+, D^- \}, \end{aligned}$$

in which $i/n = F_A(X_i)$ and $(i-1)/n = F_A(X_i - \epsilon)$ are the values in the neighborhood of each discontinuity of the ECDF, which resembles a staircase.

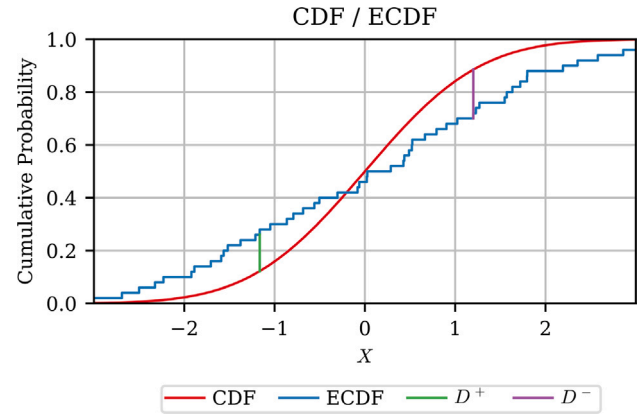


Fig. 1. A hypothetical assessment of the D^+ and D^- statistics, which allow to trivially determine the D statistic.

Fig. 1 illustrates a hypothetical assessment of the D statistic. Meanwhile, Algorithm 1 describes a pseudocode for its calculation given an unordered sample with n observations: it has memory and time complexities of $O(n)$ and $O(n \log n)$, respectively, being dominated by the sorting of the input sample.

Algorithm 1 Computation of the KS distance

Require: $A = (X_1, X_2, \dots, X_n)$, the input sample

```

1:  $D^+ = -\infty$ 
2:  $D^- = -\infty$ 
3:  $\text{sort}(A)$ 
4: for all  $i$  from 1 to  $n$  do
5:    $D^+ = \max\{D^+, i/n - F(X_i)\}$ 
6:    $D^- = \max\{D^-, F(X_i) - (i-1)/n\}$ 
7:  $D = \max\{D^+, D^-\}$ 

```

Considering the just presented panorama, a problem formulation to represent our research goal can be established: given the CDF of any reference univariate numerical distribution and a possibly infinite input stream of numerical observations, how to assess the D statistic of the KS test at any instants during input processing, while balancing computational costs and accuracy?

3. Related work

The literature [14] features an algorithm for calculating the KS test in linear time, which uses three auxiliary vectors and a bucket structure to store sample information and then obtain the exact value of the D statistic. Its notable efficiency is due to the fact that the method does not sort the input sample, but process all its elements in a two-pass fashion. Furthermore, it uses a larger amount of memory compared to its straightforward counterpart (Algorithm 1), albeit by a constant multiplicative factor. Later, a similar algorithm to approximate the KS distance was presented [9], also with linear time complexity but with a memory complexity of $O(m)$, being m a parameter regarding the number of buckets to be used as well as the accuracy of the approximation made.

More recently Lall [15] presented an algorithm to perform the KS test on data streams, whose processing should be done in an online fashion and whose time and memory complexities would ideally be constant for each data point. The algorithm is based on the use of any data structure for estimation (sketch) of sample quantiles, which also allows approximating the “inverse” ECDF as well as the KS distance, having a time and space complexity of $O(\sqrt{n} \log n)$ for a reliable estimate, as suggested by the author.

Table 1

Summary of the literature review, organized as a timeline. Time and memory complexities regard a hypothetical scenario in which n is the input size, m is a parameter of the method with no attachment to n , and the method should answer a query respective to its current condition after each input unit (batch, mini-batch, or point) is processed.

Year	Citation	Approximative	Online	Granularity	Time complexity	Memory complexity
1977	[14]	No	No	Batch	$O(n)$	$O(n)$
1978	[9]	Yes	No	Batch	$O(n)$	$O(m)$
2015	[15]	Yes	Yes	Pointwise	$O(\sqrt{n} \log n)$	$O(\sqrt{n} \log n)$
2016	[12]	Yes	Yes	Pointwise	$O(n \log n)$	$O(n)$
2017	[10]	Yes	Yes	Mini-Batch	$O(n)$	$O(m)$
2019	[28]	Yes	Yes	Pointwise	$O(nm)$	$O(m)$

Another approach, called IKS [12], is based on a treap, which is a randomized binary search tree. This one performs the update operations in time $O(\log n)$ in the average case, and queries the value of the KS distance in $O(1)$. This method can be considered similar to the one proposed here with respect to its computational cost. However, its estimation of the value of D is less straightforward and more computationally expensive in the worst case. Moreover, it stores all input observations, forbidding its use for arbitrarily large samples without any data reduction preprocessing (e.g., sampling).

Another way to perform the one-sample KS test for streams was proposed by Nguyen [10], who was the first to report tests with very large samples, aiming to provide a more realistic notion of the results in the context of Big Data. In the following year, the same author extended the just mentioned work, and then could perform the two-sample KS test [11]. Later, the first work using the KS test to detect recurrent changes in streams was presented [16].

Other less closely related works, considering the lack of use of the KS test, include a variety of methods for unsupervised drift detection [17]: some of which rely on distinct premises, as Bayesian Statistics [18]; others employ creative *ad hoc* testing procedures [19]. At last, there are some approaches whose connection to ours is even weaker but can still be mentioned: passive methods for handling concept drift, which avoid its explicit detection as active methods do [20], and supervised methods [21,22].

The calculation of quantiles in statistics is widely used, not only in computing, but in many different areas of knowledge. In the context of the KS test, quantiles are especially useful, as they represent the inverse of a CDF, or ECDF in the case of sample quantiles, and can be used to obtain the value of the KS distance [15]. In this regard, the pioneering GK algorithm [23] for online quantile estimation is capable of dealing with Big Data, making a memory usage of $O(\frac{1}{\epsilon} \log(\epsilon n))$ to answer queries with an accuracy of ϵn . This algorithm does not need to know the sample size a priori, which is a desirable feature for applications which have unbounded data streams as input. DUMIQE [24] and iQPres [25] also have this property, but only assess predefined quantiles.

Another important work introduced the Quantile Digest or Q-Digest [26], which originally focused on quantiles in a discrete sample space of size U . Q-Digest uses $O(\frac{1}{\epsilon} \log U)$ memory for its data structure: a binary tree with nodes representing variable size buckets whose size is kept approximately equal by means of merge and split operations of these. In a comparison, GK handles more varied and sparse inputs better than Q-Digest, which is superior in the opposite scenario, i.e. narrow, dense sample spaces [27]. More recently another algorithm was presented for quantile estimation in streams, the DDSketch [28], which can be considered a state-of-the-art data structure for this purpose.

Finally, Table 1 is a condensation of the literature review just detailed. Just works which we considered closely related to ours were included. They are sorted according to the year of publication, offering a chronological perspective of them.

4. Methodology

In this section we describe GreedyKS, the proposed algorithm for calculating the value of the D statistic of the KS test for one-dimensional data, presenting its pseudocode, complexity and particularities. As some of its predecessors [9,14], it is incremental and approximative, but sets itself apart by these being able to alternate between input processing and answering queries about the D statistic at any instant. Its single-parameter design allows an easier and more transparent setup compared to other relatable methods [28] but also better control over computational costs before some of its rivals [12].

GreedyKS assigns each input observation to one of the elements of a bucket array, keeping for a certain bucket with index b the following information: the greatest ($most_b$) and the lowest ($least_b$) values of the CDF of reference for its elements, as well as the element count (num_b). Apart from the distribution of reference, the number of buckets used (m) is the only parameter of the method, which can be significantly smaller than the number of observations eventually processed.

The CDF of reference itself is used to determine the bucket each observation is associated with. The method maintains two variables DP and DN , correspondingly associated to D^+ (positive difference) and D^- (negative difference) which were illustrated in Fig. 1, to record the extent of the largest gaps to the ECDF from the CDF and *vice-versa*, respectively. For each new element processed, the algorithm updates the three attributes of the corresponding bucket, calculates the differences between ECDF and CDF in the region corresponding to the bucket where the new element was inserted and, if necessary, updates the variables DP and DN greedily.

It should be noted that updating the value of these variables taking into account only the last modified bucket induces a relatively poor accuracy. Targeting this issue, the indexes of the last buckets which maximized these variables are also kept so they can be reevaluated in future iterations. This has a negligible (constant) impact on the algorithmic complexity of the method, which remains estimating D greedily, but leads to a desirable improvement on its reliability. The importance of such an action becomes clearer considering that the update of a single bucket reflects on all buckets, increasing or decreasing the gap between CDF and ECDF in the region they represent, as shown in Appendix A.

When iterating over the input, each new element can be assigned to any of the buckets. To calculate the differences between ECDF and CDF relative to a bucket, how many elements were placed in preceding buckets has to be known. In order to optimize such a query, a data structure for the prefix sums [29] was used in order to obtain the number of elements positioned up to a given bucket: e.g., Fenwick tree [30] (also known as Binary Indexed Tree, BIT), segment tree [31], or square-root decomposition [32]. In this work, the BIT was used because of its update and query operations whose computational cost grows logarithmically with the size of the array whose prefixes are to be assessed.

For the assessment and analysis of the algorithmic complexity of the proposed method, Algorithm 2 describes the pseudocode of an

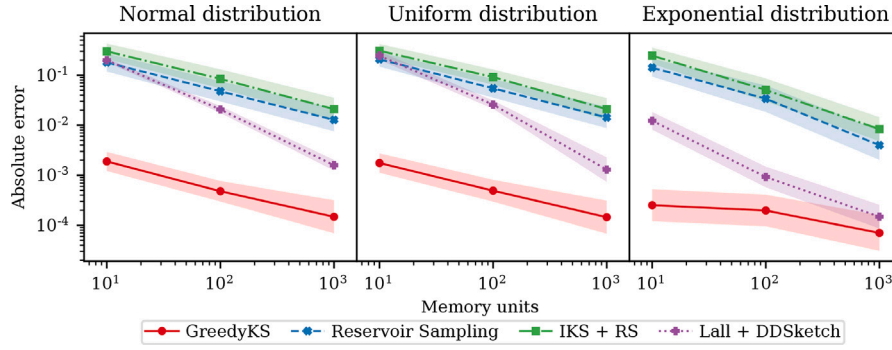


Fig. 2. Absolute error of estimations of the D statistic when varying available memory.

iteration of GreedyKS during input processing. The computational cost of the algorithm is predominantly defined by updating and querying the prefix sums (lines 4, 7, 8, 12, and 13 of Algorithm 2), each of which taking $O(\log m)$ steps. The operations in the remaining lines are atomic, having a constant time cost. Therefore each iteration has a time complexity of $O(\log m)$, so that both efficiency and precision are attached to the chosen number of buckets, but not to the input size. The estimated value of the D statistic can be obtained at any instant in constant time as the result of $\max\{DN, DP\}$.

Algorithm 2 GreedyKS iteration

Require: x , the element to be processed
Require: n , the number of elements processed so far
Require: $F(\cdot)$, the CDF of reference
Require: $m, num_b, most_b$, and $least_b$, as previously defined

```

1:  $f = F(x)$ 
2:  $n = n + 1$ 
3:  $b = \lfloor f \cdot m \rfloor$ 
4:  $num_b = num_b + 1$                                 ▷ prefix sum update
5:  $most_b = \max\{most_b, f\}$ 
6:  $least_b = \min\{least_b, f\}$ 
7:  $DN = least_{oldN} - (\sum_{i=0}^{oldN-1} num_i)/n$           ▷ prefix sum query
8:  $DN' = least_b - (\sum_{i=0}^{b-1} num_i)/n$               ▷ prefix sum query
9: if  $DN' > DN$  then
10:    $DN = DN'$ 
11:    $oldN = b$ 
12:  $DP = (\sum_{i=0}^{oldP} num_i)/n - most_{oldP}$           ▷ prefix sum query
13:  $DP' = (\sum_{i=0}^b num_i)/n - most_b$                 ▷ prefix sum query
14: if  $DP' > DP$  then
15:    $DP = DP'$ 
16:    $oldP = b$ 

```

5. Experimental evaluation

The proposed algorithm was experimentally evaluated using synthetic and real data, rivaling with alternative methods. The experiments, whose source code is publicly available,¹ were executed using a computer with an Intel(R) Core(TM) i7-9750H CPU with 16 GB RAM running an Ubuntu 21.10 operating system. The tests with synthetic data were adapted from an experimental protocol previously established in the literature [15]. In this regard, unless otherwise stated: 100 independent samples were generated and the averages of the results were reported; each sample had 10^4 observations following a Normal, Uniform or Exponential distribution; the available memory for each method was of the order of 10^2 memory units (e.g., $m = 10^2$ for GreedyKS), with each unit corresponding to one observation; the means

of the reference and sample distributions were 1.0 and 1.1 respectively (i.e., the difference between them was of 10^{-1}), while the standard deviation of both was 1.0. The experiments were purposely carried out using similar distributions, in order to observe the behavior of the methods before small distinctions. After all, when the distributions are unlike, it is relatively easy to assert this even with substantial resource constraints.

In addition to GreedyKS, the following methods were also considered:

- Reservoir Sampling (RS) [8] - iteratively sample the input and calculate the exact value of the D statistic for the subsample. It is considered a baseline solution.
- IKS + RS [8,12] - iteratively sample the input filtering it to IKS, which by default stores all its input, assuming the absence of memory restrictions.
- Lall + DDSketch [15,28] - a method which relies on any quantile sketch, combined with one of the state-of-the-art data structures for this purpose.

5.1. Estimation error

In the first set of experiments, the absolute error (the module of the difference between the true, exact value and the estimation) of the D statistic of the one-sample KS test obtained by each method in question was measured. Such values were computed after the incremental processing of a given input sample. Several runs were made, targeting to depict scenarios based on varied statistical distributions and a set of numerical parameters: available memory (i.e., number of memory units), difference between sample mean (or standard deviation) and that of the reference distribution, and sample size.

Fig. 2 illustrates a comparison focused on the restriction of available memory. In general, the methods showed a similar behavior for the three types of distribution. The only exception was Lall + DDSketch, which presented a performance approximately 1 order of magnitude better in the scenario of the Exponential distribution, what also happened in the next experiments. The methods showed an improvement trend (smaller error) as available memory increases, what could be expected. IKS + RS had a slightly worse performance compared with pure Reservoir Sampling possibly because it aggregates the errors of both techniques, what is supported by the fact that the relative performance of both methods was the same in the next experiments. GreedyKS exhibited the lowest error overall.

It was also assessed how the methods respond to differences in distribution parameters. Fig. 3 illustrates this with respect to differences in the distribution mean. All methods demonstrated to be quite stable in general, with no pronounced variation of the results in the entire range of values considered. Regardless of that, GreedyKS had the superior performance by a wide margin.

The results obtained when varying the difference between the standard deviations of the sample and reference distributions, depicted in

¹ https://github.com/moretoknow/greedyks_experiments

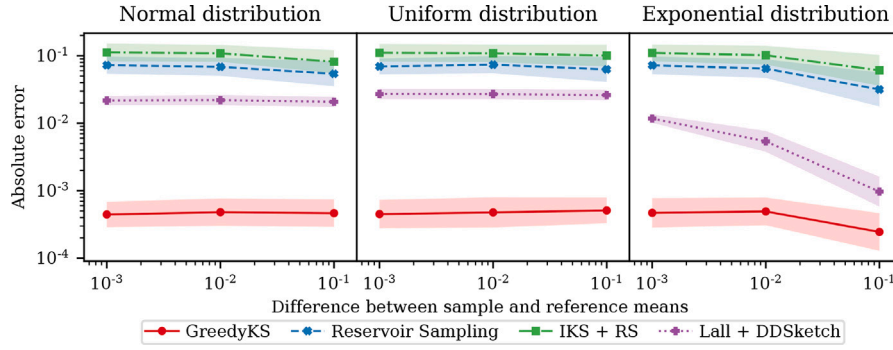


Fig. 3. Absolute error of estimations of the D statistic when varying the difference between the means of the reference and sample distributions.

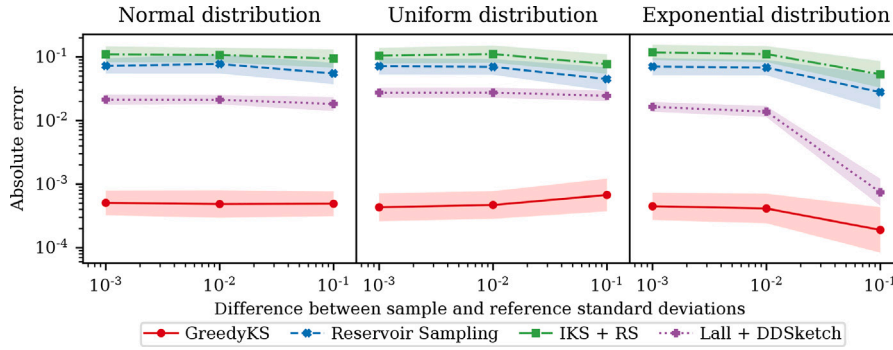


Fig. 4. Absolute error of estimations of the D statistic when varying the difference between the standard deviations of the reference and sample distributions.

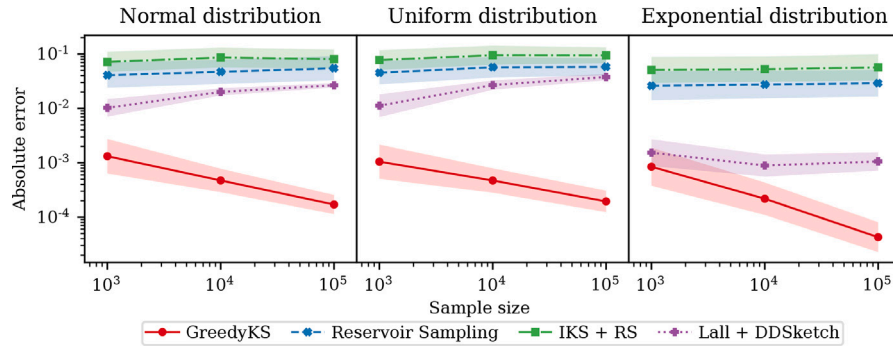


Fig. 5. Absolute error of estimations of the D statistic when varying input sample size.

Fig. 4, were very similar to those of the previous experiment, regarding the difference between means: same ranking of all methods in the three scenarios, with GreedyKS again setting itself apart from the others. Such insensitivity to changes in distribution parameters is an interesting trait shared by all considered alternatives. Still in this regard, for a clearer assessment of the results about differences between standard deviations, in this experiment the mean of both reference and sample distributions was 1.0.

In Fig. 5 the results when sample size was varied are illustrated, to observe how the methods react to data volume increase. Compared to the previous experiments, there was no difference in the ranking of the methods. However, GreedyKS once more showed an improvement trend, indicating that with more observations seen in the total its approximation of the value of D becomes more precise, while the others methods showed a stabilizing trend.

5.2. Efficiency

The second set of experiments regarded the time required by each method for incrementally processing a given input sample, taking into

account updates respective to each observation as well as interposed queries for the estimated value of D . In this set of experiments only the Normal distribution was considered, since variations in this aspect had a negligible impact in processing time. The experiments were performed varying the number of observations processed between successive queries (i.e., the period of the queries), size of the input sample and number of available memory units. A default period of the queries of 1 was used unless when this experiment parameter was being varied.

Fig. 6 illustrates the obtained results. In the first scenario two kinds of behavior are exhibited by the methods: the time required by GreedyKS and IKS + RS practically remained the same regardless of the query period, differing between each other only by a constant; on the other hand, Lall + DDSketch and Reservoir Sampling showed an improvement trend as the query period increased. This is explained by the fact that only the first two employ an eager strategy with respect to the estimation of D , while the last two perform this evaluation lazily, delaying its processing as much as possible until a query is realized, so that it dominates the overall computational cost.

In the second and third scenarios sample size and available memory were varied, respectively. All methods had an increase in total time as

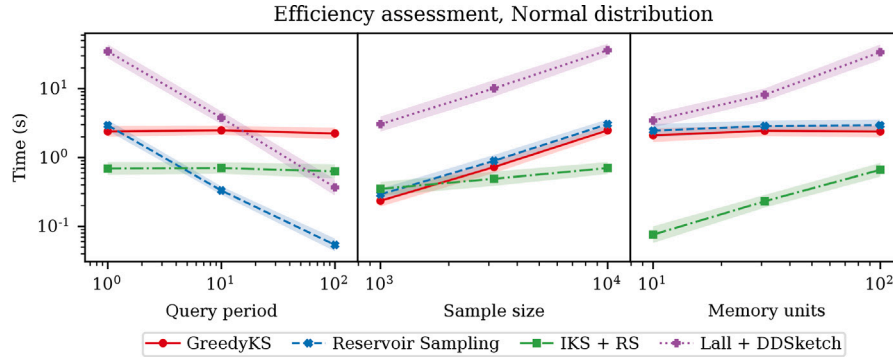


Fig. 6. Time, in seconds, required for incrementally processing a given input sample, taking into account answering queries for estimations of the D statistic.

Table 2

Effectiveness in concept drift detection when varying available memory. The values shown in each cell represent, for 100 runs: the number of premature detections, the number of missed detections, and the average number of observations needed for correct detections.

	Memory units	GreedyKS	Exact KS	RS	IKS+RS	Lall+ DDSketch
Normal	10 ¹	12/0/1778.3	14/0/1523.9	15/78/2638.1	19/66/4144.3	100/0/-
	10 ^{1.5}	10/0/1665.7	7/0/1436.8	17/72/4112.1	23/61/2943.6	100/0/-
	10 ²	10/0/1625.0	10/0/1518.7	14/73/3309.5	19/64/2848.4	51/0/517.0
Uniform	10 ¹	10/0/2231.6	13/0/1770.7	13/82/5173.0	25/67/3624.4	100/0/-
	10 ^{1.5}	12/0/2212.1	11/0/1888.1	11/81/2941.2	23/62/2915.7	100/0/-
	10 ²	6/0/1837.1	6/0/1763.1	12/76/4000.2	20/64/2790.9	28/0/423.4
Exponential	10 ¹	6/0/646.5	8/0/569.5	15/80/3245.0	19/71/3174.4	100/0/-
	10 ^{1.5}	6/0/583.8	6/0/534.2	13/78/3387.3	32/55/2190.4	100/0/-
	10 ²	6/0/571.5	7/0/556.3	12/59/2959.0	19/47/2992.9	20/0/454.6

the input sample grew, as expected. However, the rate of such increase for IKS + RS is lower compared to the other methods, and that happens because thanks to reservoir sampling not all input observations lead to updates of IKS treap, which are the most computationally expensive operations of this method. About varying the available memory, GreedyKS and Reservoir Sampling had an almost imperceptible change while IKS + RS and Lall + DDSketch have a much more noticeable shift of their results. Considering the trends exhibited in both scenarios, GreedyKS could be preferred for situations in which a larger amount of memory was available, even if this was matched by a larger input sample.

5.3. Effectiveness in concept drift detection

In a third set of experiments, it was assessed how GreedyKS and its alternatives operate when the provided incremental input is a concatenation of two subsamples with similar but different distributions. The ultimate goal was to detect concept drift (i.e. the switch between distributions) as soon as possible, considering the number of observations of the second portion required for this. The input sample was divided as follows: the first 10% of the observations followed the reference distribution while the remaining 90% followed the same distribution but with slightly different parameters. Variations of the same parameters of the experiments about estimation error (Section 5.1) were assessed. Baseline results provided by the regular, “Exact” KS test are also reported.

All methods followed the same protocol: after each observation was processed, if the number of past observations was greater than the number of memory units available, it was checked if a concept drift occurred based on a KS test with $p = 0.01$ considering the estimated value of D . Using the same threshold for all methods was considered reasonable since they rely on the same principle, despite their particularities [33]. The entire procedure had three possible outcomes: drift was signaled while observations of the first portion of the sample are processed (i.e., a false positive), or the same but for observations of the second portion (i.e., a true positive), or no drift was signaled at all (i.e., a false negative). The cells of the result tables shown

next present the number of times the undesired outcomes (first and third) happened, and when the second outcome happened, the average number of observations of the second portion that were processed before drift was detected.

Table 2 shows the results based on varying available memory. The methods had similar performances for the three distributions, with the Exponential one enabling the best results, what also happened in experiments to follow as well. In most runs Reservoir Sampling and IKS + RS did not detect any drift, and when they correctly did it, they were slower than all other methods doing so. On the other hand, Lall + DDSketch was exaggeratedly sensitive under the most strict conditions, leading to numerous false positives. GreedyKS had the best results, which were quite similar to those of Exact KS.

The results based on varying the size of the input sample are presented in Table 3. Once again, Lall + DDSketch had a substantial number of premature detections. About Reservoir Sampling and IKS + RS, it can be noticed that with an increase of 10 times in sample size, their mean time to detect drift was multiplied by a factor between 4 and 10, while for GreedyKS and Exact KS the respective multiplicative factor would be between 2 and 3.

Table 4 shows the results obtained while varying the difference between the means of the reference distribution and that of the second portion of the input sample. As expected, the performance of all methods improves as the difference increases, since concept drift becomes more evident. Lall + DDSketch had the best performance with respect to how fast it signaled the drifts, but on the other hand it had a high false positive rate just as before. GreedyKS had the worst performance overall when the difference was minimal (and the best when it was maximal), but even in this case it mirrored Exact KS.

At last, Table 5 presents the results of the tests based on the variation of the difference between the standard deviations of the reference distribution and that of the second portion of the input sample. The results are similar to the tests varying the difference between means. However, they allow to suggest that concept drifts with respect to standard deviation are more subtle, considering the general increase in false negative rate and detection speed. One more time GreedyKS mirrored

Table 3

Effectiveness in concept drift detection when varying sample size. The values shown in each cell represent, for 100 runs: the number of premature detections, the number of missed detections, and the average number of observations needed for correct detections.

	Sample size	GreedyKS	Exact KS	RS	IKS+RS	Lall+ DDSketch
Normal	$10^{3.5}$	3/0/1051.4	3/0/1040.0	7/75/1074.3	10/66/980.0	7/0/562.7
	10^4	10/0/1604.2	11/0/1539.1	12/76/2639.2	11/74/3114.2	39/0/389.7
	$10^{4.5}$	8/0/2421.9	9/0/2321.7	16/68/9747.6	26/58/7619.6	98/0/180.5
Uniform	$10^{3.5}$	5/7/1282.9	6/5/1268.5	5/81/1113.1	14/63/1010.0	14/0/558.3
	10^4	6/0/1951.6	6/0/1881.3	9/76/2822.1	19/64/2413.6	33/0/422.7
	$10^{4.5}$	14/0/2590.3	15/0/2486.7	18/64/9033.1	35/54/4206.7	100/0/-
Exponential	$10^{3.5}$	2/0/372.1	2/0/364.6	5/62/1230.0	14/63/1011.9	5/0/327.5
	10^4	4/0/572.3	4/0/553.0	9/65/3463.6	30/48/3404.5	21/0/424.6
	$10^{4.5}$	9/0/952.8	10/0/924.8	13/57/9471.5	36/45/7799.5	74/0/443.8

Table 4

Effectiveness in concept drift detection when varying the difference between the mean of the reference distribution and that of an anomalous portion of the input sample. The values shown in each cell represent, for 100 runs: the number of premature detections, the number of missed detections, and the average number of observations needed for correct detections.

	Means Diff.	GreedyKS	Exact KS	RS	IKS+RS	Lall+ DDSketch
Normal	10^{-2}	7/83/2801.6	9/79/3592.6	12/80/3228.5	23/60/4069.4	53/0/993.6
	$10^{-1.5}$	7/56/3865.4	6/52/4055.3	9/78/2167.5	18/72/2700.6	40/0/728.5
	10^{-1}	10/0/1537.1	10/0/1490.4	9/71/3076.8	22/55/2911.0	42/0/493.1
Uniform	10^{-2}	6/83/2646.0	6/80/2640.1	15/75/1974.5	26/58/2488.4	28/0/586.0
	$10^{-1.5}$	11/49/5277.6	8/49/5137.4	10/76/3721.1	21/67/2107.4	39/0/645.7
	10^{-1}	6/0/1872.4	6/0/1796.3	12/73/3164.2	18/65/3908.1	39/0/498.3
Exponential	10^{-2}	9/76/4318.7	9/73/3905.0	9/83/3751.8	19/70/1884.0	24/1/1959.9
	$10^{-1.5}$	13/0/2547.4	10/0/2432.5	15/77/1082.6	19/69/3447.8	29/0/1515.9
	10^{-1}	13/0/574.7	13/0/553.9	16/61/3228.4	30/42/3175.0	24/0/455.6

Table 5

Effectiveness in concept drift detection when varying the difference between the standard deviation (SD) of the reference distribution and that of an anomalous portion of the input sample. The values shown in each cell represent, for 100 runs: the number of premature detections, the number of missed detections, and the average number of observations needed for correct detections.

	SDs Diff.	GreedyKS	Exact KS	RS	IKS+RS	Lall+ DDSketch
Normal	10^{-2}	10/82/2556.2	9/82/2728.2	11/77/3588.7	21/61/2860.1	42/0/919.8
	$10^{-1.5}$	11/75/5049.4	12/71/4925.5	11/84/1610.2	25/62/2693.9	45/0/835.1
	10^{-1}	6/0/2734.6	8/0/2614.4	11/80/2407.8	27/62/2446.9	49/0/577.2
Uniform	10^{-2}	7/85/3923.8	8/82/3430.9	12/78/3602.4	24/63/4368.3	34/0/667.9
	$10^{-1.5}$	9/50/6083.8	8/41/6246.8	11/81/3488.1	17/72/2612.1	36/0/489.5
	10^{-1}	6/0/1494.4	6/0/1449.9	9/81/3314.4	27/59/2195.1	36/0/346.0
Exponential	10^{-2}	4/82/3124.1	4/80/3683.9	14/73/2042.2	27/59/1971.1	35/0/1540.7
	$10^{-1.5}$	12/0/3031.1	12/0/2901.2	14/73/3424.4	17/68/2861.9	33/0/992.1
	10^{-1}	8/0/668.2	8/0/661.1	11/73/2623.6	19/61/3922.8	34/0/530.4

Exact KS, being too passive for the smallest difference considered but excelling in the opposite situation.

5.4. Online assessment of real data

In order to complete the experimental assessment of GreedyKS, its performance in a more realistic setting was also analyzed. The previous experiments required operating under controlled conditions with respect to concept drift, which was artificially produced, consequently assuring an undisputed ground truth which was used for rating the considered methods for the proposed tasks. This convenience would be absent if data spontaneously produced by some uninduced process was to be used for the same sort of comparison. However, overcoming such an enhanced challenge is of great interest considering the diversity of practical applications of drift detection, and the need to decide which method would be used in each of them. A description of a dataset with the just mentioned characteristics as well as an approach for its evaluative use are indicated next.

A common call center can be modeled as a dynamic system given that its workload may fluctuate according to numerous external factors,

such as: an ample interruption of a service, as well as its resumption; time-related features (e.g., present month, day of week, hour, holidays). Perceiving when there is a significant change of such workload enables decisions regarding service maintenance and staff management. An approach for this can rely on modeling call occurrence as a Poisson process [34] whose parameters are stable within blocks of time with a reasonable size, like one-hour intervals [35], so that the spans between subsequent calls would follow an Exponential distribution.

Based on the aforementioned modeling, drift detection could be done as follows: in an hourly basis, use a KS test to verify if the collection of calls since a checkpoint follows a previously established Exponential distribution; if the test fails, signal the drift and redefine both the checkpoint and the distribution parameters according to all calls from the latest hour. This “mini-batch” approach could be improved by increasing its granularity, up to the point of performing the KS test after each new call, targeting promptest detection. GreedyKS and its rivals were tested in this role, and their outputs were compared to that of the mini-batch method, taken as ground truth: since each of these methods could provide an arbitrarily-sized list of instants in which they signaled drifts, their resemblance was evaluated using

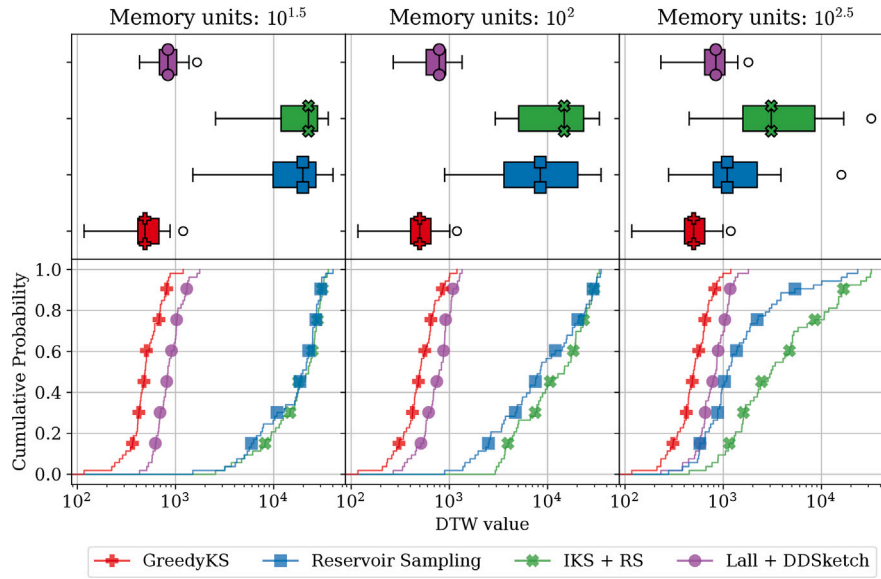


Fig. 7. Empirical distribution of DTW values between drift points defined by a mini-batch approach (ground truth) and those inferred by GreedyKS and its rivals.

Dynamic Time Warping (DTW) [36,37], a well-established algorithm for assessing the similarity between time series.

A popular dataset of a bank call center was used in this regard.² It comprises an entire year, totaling 444,448 call records. For the sake of statistical validation, these records were partitioned into 53 subsets according to the week of the year in which they happened. Each of these subsets was processed as an independent input sample. The results are illustrated in Fig. 7. The superiority of GreedyKS can be perceived by the interpretation that it had the best balance between detecting drifts as early as possible while avoiding being prone to false alarms, since both aspects would be penalized in the DTW evaluation. Moreover, according to Wilcoxon signed-rank tests with a significance level (α) of 0.01, GreedyKS significantly prevailed over its rivals in all three memory scenarios considered, although such fact was more pronounced in the one with stricter constraints.

A closer look at the behavior of the compared methods could provide interesting insights, complementing the results just described. Based on this idea, we considered this scenario: phone calls realized during the first work hour of the first workday of the dataset (i.e., from 8:00 to 8:59 of 1999-01-04) would be used for fitting a reference Exponential distribution; then, it would be used to detect the earliest drift point of subsequent phone calls. Such results for GreedyKS and its rivals are shown in Table 6. For reference, a mini-batch approach detected the drift between 20:00 and 21:00, coinciding with GreedyKS. Hypothetically, such a drift would be related to a decrease in call frequency at the end of the day. Meanwhile, Lall + DDSketch detected the drift too early while IKS + RS and Reservoir Sampling were days overdue.

Moreover, Fig. 8 enables the visual assessment of goodness-of-fit of the subsamples up to the drift points indicated by each method. These are consistent with the results of Table 6: the methods which tardily signaled the drift correspond to curves which are the most distant to the reference ones; on the other hand, the curve regarding Lall + DDSketch appears to match too well the reference ones, contradicting the predicted difference between distributions.

Table 6

Results of the experiment of drift detection of call records starting from 8:00 of 1999-01-04. The value of the D statistic considered a reference Exponential distribution fitted to first hour subsample and all data up to the detected drift point.

Methods	Results	
	Drift point	D statistic
GreedyKS	1999-01-04 21:41:26	0.039804
IKS + RS	1999-01-09 18:31:35	0.118255
Lall + DDSketch	1999-01-04 11:40:41	0.040808
Reservoir Sampling	1999-01-06 16:08:55	0.094218

6. Conclusion

This paper regarded the problem of efficiently performing the Kolmogorov–Smirnov test in an online fashion, targeting Big Data applications. We presented GreedyKS, an incremental algorithm for approximating the one-sample version of the test. Its original implementation is publicly available.³ This method combines the conventional test calculation with any auxiliary data structure for prefix sum assessment, enabling its realization even under severe constraints regarding computational resources. Furthermore, we showed that the performance of our algorithm was superior not only to that of a standard approach such as sampling but also to those of more refined methods: this was confirmed through numerous experiments, varied by the use of synthetic and real data, and also parameters as memory availability. According to the results of such tests, GreedyKS was especially advantageous compared to its rivals for larger input samples, evidencing its applicability in such a realistic condition.

The just mentioned subject of realistic applications can be further explored by projecting concrete uses of our findings. In a similar fashion to the reported experiment based on call center records, that approach could be adapted to other scenarios which also allow a reference distribution to be established *a priori*: these could be featured in various areas, like economy, operations research, or astronomy. Still in the same regard, another situation in which GreedyKS could be most advantageous is to be implemented in embedded systems, considering its performance when required to exhibit a minimal memory footprint.

² <https://iew.technion.ac.il/serveng/callcenterdata/index.html> (accessed in 2022/05/25)

³ <https://github.com/moretoknow/driftwatch>

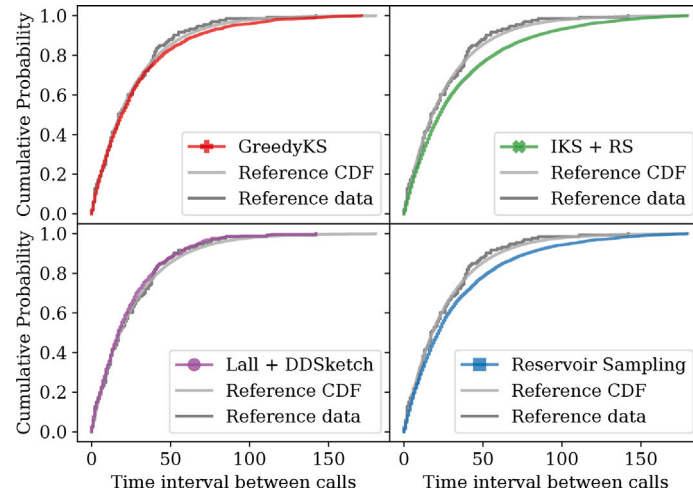


Fig. 8. ECDF plots of the subsample of calls from 8:00 to 8:59 of 1999-01-04, the corresponding fitted Exponential distribution, and the subsamples determined by the drift points indicated in Table 6..

There are some points for improvement that still persist. We believe the error analysis deserves to be further investigated, as this could enable to establish stricter bounds in this regard as well as improving approximation accuracy with respect to the classic, exact, batch KS test. Another idea would be to employ other data structures and algorithmic techniques for an even more sophisticated and efficient version of the method. Besides such hypothetical developments of GreedyKS, other related methods could also be considered: for example, a similar solution to the two-sample version of the KS test could also be appreciated.

CRedit authorship contribution statement

Douglas O. Cardoso: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Review & editing, Visualization, Resources, Supervision, Project administration, Funding acquisition. **Thalis D. Galeno:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Review & editing, Visualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Source code and data to reproduce the reported experiments can be found at <https://github.com/moretoknow/driftwatch>, https://github.com/moretoknow/greedyks_experiments, and <https://iew.technion.ac.il/serveng/callcenterdata/index.html>.

Funding

This work was supported by the Carlos Chagas Filho Foundation for Research Support of Rio de Janeiro State (FAPERJ), Brazil [grant number 200693 to T.D.G].

Appendix A. GreedyKS dynamics

This appendix establishes some propositions regarding the dynamics of buckets updates during GreedyKS runtime. In this regard, consider a hypothetical assignment of an observation to a bucket b . Let $least_i$ and $least_i^*$ be the lowest values of the CDF of reference for the elements of a bucket i before and after the assignment, respectively. Also, let $most_i$ and $most_i^*$ be the greatest values of the CDF of reference for the elements of a bucket i before and after the assignment, respectively. Likewise, let num_i and num_i^* be the numbers of elements of a bucket i before and after the assignment, respectively.

Remark 1. The sum of elements up to any bucket $a < b$ would remain unaltered but the total number of elements would be incremented, so that

$$\frac{\sum_{i=0}^a num_i^*}{n+1} = \frac{\sum_{i=0}^a num_i}{n+1} \cdot \frac{n}{n} = \frac{(\sum_{i=0}^a num_i)(n+1-1)}{n(n+1)} = \frac{(\sum_{i=0}^a num_i)(n+1) - \sum_{i=0}^a num_i}{n(n+1)} = \frac{\sum_{i=0}^a num_i}{n} - \frac{\sum_{i=0}^a num_i}{n(n+1)}.$$

Remark 2. The sum of elements up to any bucket $c \geq b$ would be incremented like the total number of elements, so that

$$\frac{\sum_{i=0}^c num_i^*}{n+1} = \frac{1 + \sum_{i=0}^c num_i}{n+1} \cdot \frac{n}{n} = \frac{n + (n+1-1) \sum_{i=0}^c num_i}{n(n+1)} = \frac{(n+1) \sum_{i=0}^c num_i}{n(n+1)} + \frac{n - \sum_{i=0}^c num_i}{n(n+1)} = \frac{\sum_{i=0}^c num_i}{n} + \frac{n - \sum_{i=0}^c num_i}{n(n+1)}.$$

Proposition 1. The assignment increases the difference to the CDF from the ECDF in the region of any bucket $a < b$ by

$$\frac{\sum_{i=0}^{a-1} num_i}{n(n+1)}.$$

Proof. Considering line 8 of Algorithm 2 and Remark 1:

$$least_a^* - \frac{\sum_{i=0}^{a-1} num_i^*}{n+1} = \left(least_a - \frac{\sum_{i=0}^{a-1} num_i}{n} \right) + \frac{\sum_{i=0}^{a-1} num_i}{n(n+1)}. \quad \square$$

Proposition 2. *The assignment decreases the difference to the ECFD from the CDF in the region of any bucket $a < b$ by*

$$\frac{\sum_{i=0}^a num_i}{n(n+1)}.$$

Proof. Considering line 13 of Algorithm 2 and Remark 1:

$$\frac{\sum_{i=0}^a num_i^*}{n+1} - most_a^* = \left(\frac{\sum_{i=0}^a num_i}{n} - most_a \right) - \frac{\sum_{i=0}^a num_i}{n(n+1)}. \quad \square$$

Proposition 3. *The assignment decreases the difference to the CDF from the ECFD in the region of any bucket $c > b$ by*

$$\frac{n - \sum_{i=0}^{c-1} num_i}{n(n+1)}.$$

Proof. Considering line 8 of Algorithm 2 and Remark 2:

$$least_c^* - \frac{\sum_{i=0}^{c-1} num_i^*}{n+1} = \left(least_c - \frac{\sum_{i=0}^{c-1} num_i}{n} \right) - \frac{n - \sum_{i=0}^{c-1} num_i}{n(n+1)}. \quad \square$$

Proposition 4. *The assignment increases the difference to the ECFD from the CDF in the region of any bucket $c > b$ by*

$$\frac{n - \sum_{i=0}^c num_i}{n(n+1)}.$$

Proof. Considering line 13 of Algorithm 2 and Remark 2:

$$\frac{\sum_{i=0}^c num_i^*}{n+1} - most_c^* = \left(\frac{\sum_{i=0}^c num_i}{n} - most_c \right) + \frac{n - \sum_{i=0}^c num_i}{n(n+1)}. \quad \square$$

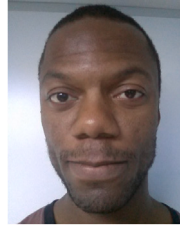
Appendix B. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.jocs.2023.101959>.

References

- [1] H. Hu, M. Kantardzic, T.S. Sethi, No free lunch theorem for concept drift detection in streaming data classification: A review, *Wiley Interdisc. Rev. Data Min. Knowl. Discov.* 10 (2) (2020) e1327.
- [2] D. Kifer, S. Ben-David, J. Gehrke, Detecting change in data streams, in: *Proceedings of the Thirtieth International Conference on Very Large Data Bases - Volume 30*, in: VLDB '04, VLDB Endowment, Toronto, Canada, 2004, pp. 180–191.
- [3] J. Grzyb, J. Klikowski, M. Woźniak, Hellinger distance weighted ensemble for imbalanced data stream classification, *J. Comput. Sci.* 51 (2021) 101314.
- [4] K.C. Claffy, G.C. Polyzos, H.-W. Braun, Application of sampling methodologies to network traffic characterization, *ACM SIGCOMM Comput. Commun. Rev.* 23 (4) (1993) 194–203, <http://dx.doi.org/10.1145/167954.166256>.
- [5] D. Barbosa, J.P. Barraca, B. Carvalho, D. Maia, Y. Gupta, S. Natarajan, G.L. Roux, P. Swart, A cyber infrastructure for the SKA telescope manager, in: G. Chiozzi, J.C. Guzman (Eds.), *Software and Cyberinfrastructure for Astronomy IV*, Vol. 9913, SPIE, International Society for Optics and Photonics, Edinburgh, United Kingdom, 2016, pp. 213–224, <http://dx.doi.org/10.1117/12.2234002>.
- [6] A.M.M. Scaife, Big telescope, big data: Towards exascale with the square kilometre array, *Phil. Trans. R. Soc. A* 378 (2166) (2020) 20190060, <http://dx.doi.org/10.1098/rsta.2019.0060>, URL <https://royalsocietypublishing.org/doi/10.1098/rsta.2019.0060>.
- [7] S. Agrahari, A.K. Singh, Concept drift detection in data stream mining : A literature review, *J. King Saud Univ. - Comput. Inform. Sci.* (2021) <http://dx.doi.org/10.1016/j.jksuci.2021.11.006>.
- [8] A. Bifet, *Machine Learning for Data Streams: With Practical Examples in MOA*, in: *Adaptive computation and machine learning series*, MIT Press, Cambridge, Massachusetts, 2017.
- [9] T. Gonzalez, S. Sahni, W.R. Franta, An efficient approximate algorithm for the Kolmogorov–Smirnov and Lilliefors tests, *J. Stat. Comput. Simul.* 6 (3–4) (1978) 257–263, <http://dx.doi.org/10.1080/00949657808810194>.
- [10] H.D. Nguyen, A stream-suitable Kolmogorov–Smirnov-type test for big data analysis, 2017, [arXiv:1704.03721](https://arxiv.org/abs/1704.03721).
- [11] H.D. Nguyen, A two-sample Kolmogorov–Smirnov-like test for big data, in: Y.L. Boo, D. Stirling, L. Chi, L. Liu, K.-L. Ong, G. Williams (Eds.), *Data Mining*, in: *Communications in Computer and Information Science*, Springer, Singapore, 2018, pp. 89–106, http://dx.doi.org/10.1007/978-981-13-0292-3_6.
- [12] D.M. dos Reis, P. Flach, S. Matwin, G. Batista, Fast unsupervised online drift detection using incremental Kolmogorov–Smirnov test, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, in: KDD '16, Association for Computing Machinery, New York, NY, USA, 2016, pp. 1545–1554, <http://dx.doi.org/10.1145/2939672.2939836>.
- [13] T.D. Galeno, J. Gama, D.O. Cardoso, A sketch for the KS test for big data, in: *Anais Do IX Symposium on Knowledge Discovery, Mining and Learning (KDMiLe 2021)*, Sociedade Brasileira de Computação - SBC (In Portuguese), Brasil, 2021, pp. 8–15, <http://dx.doi.org/10.5753/kdmile.2021.17455>, URL <https://sol.sbc.org.br/index.php/kdmile/article/view/17455>.
- [14] T. Gonzalez, S. Sahni, W.R. Franta, An efficient algorithm for the Kolmogorov–Smirnov and Lilliefors Tests, *ACM Trans. Math. Software* 3 (1) (1977) 60–64, <http://dx.doi.org/10.1145/355719.355724>.
- [15] A. Lall, Data streaming algorithms for the Kolmogorov–Smirnov test, in: *Proceedings of the 2015 IEEE International Conference on Big Data (Big Data)*, in: BIG DATA '15, IEEE Computer Society, USA, 2015, pp. 95–104, <http://dx.doi.org/10.1109/BigData.2015.7363746>.
- [16] C. Raab, M. Heusinger, F.-M. Schleif, Reactive soft prototype computing for concept drift streams, *Neurocomputing* 416 (2020) 340–351, <http://dx.doi.org/10.1016/j.neucom.2019.11.111>.
- [17] R.N. Gemaque, A.F.J. Costa, R. Giusti, E.M. Santos, An overview of unsupervised drift detection methods, *WIREs Data Min. Knowl. Discov.* 10 (6) (2020) <http://dx.doi.org/10.1002/widm.1381>, URL <https://onlinelibrary.wiley.com/doi/10.1002/widm.1381>.
- [18] J. Xuan, J. Lu, G. Zhang, Bayesian nonparametric unsupervised concept drift detection for data stream mining, *ACM Trans. Intell. Syst. Technol.* 12 (1) (2021) 1–22, <http://dx.doi.org/10.1145/3420034>, URL <https://dl.acm.org/doi/10.1145/3420034>.
- [19] V.M.A. Souza, A.R.S. Parmezan, F.A. Chowdhury, A. Mueen, Efficient unsupervised drift detector for fast and high-dimensional data streams, *Knowl. Inf. Syst.* 63 (6) (2021) 1497–1527, <http://dx.doi.org/10.1007/s10115-021-01564-6>, URL <https://link.springer.com/10.1007/s10115-021-01564-6>.
- [20] M. Han, Z. Chen, M. Li, H. Wu, X. Zhang, A survey of active and passive concept drift handling methods, *Comput. Intell.* (2022) coin.12520, <http://dx.doi.org/10.1111/coin.12520>, URL <https://onlinelibrary.wiley.com/doi/10.1111/coin.12520>.
- [21] H. Guo, H. Li, Q. Ren, W. Wang, Concept drift type identification based on multi-sliding windows, *Inform. Sci.* 585 (2022) 1–23, <http://dx.doi.org/10.1016/j.ins.2021.11.023>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0020025521011439>.
- [22] R.S.M. Barros, S.G.T.C. Santos, A large-scale comparison of concept drift detectors, *Inform. Sci.* 451–452 (2018) 348–370, <http://dx.doi.org/10.1016/j.ins.2018.04.014>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0020025518302743>.
- [23] M. Greenwald, S. Khanna, Space-efficient online computation of quantile summaries, *ACM SIGMOD Rec.* 30 (2) (2001) 58–66, <http://dx.doi.org/10.1145/376284.375670>.
- [24] H.L. Hammer, A. Yazidi, Incremental quantiles estimators for tracking multiple quantiles, in: S. Benferhat, K. Tabia, M. Ali (Eds.), *Advances in Artificial Intelligence: from Theory to Practice*, in: *Lecture Notes in Computer Science*, Springer International Publishing, Cham, 2017, pp. 202–210, http://dx.doi.org/10.1007/978-3-319-60042-0_23.
- [25] K. Tschumitschew, F. Klawonn, Incremental quantile estimation, *Evol. Syst.* 1 (4) (2010) 253–264, <http://dx.doi.org/10.1007/s12530-010-9017-7>, URL <http://link.springer.com/10.1007/s12530-010-9017-7>.
- [26] N. Shrivastava, C. Buragohain, D. Agrawal, S. Suri, Medians and beyond: New aggregation techniques for sensor networks, in: *Proceedings of the 2nd International Conference on Embedded Networked Sensor Systems, SenSys '04*, Association for Computing Machinery, New York, NY, USA, 2004, pp. 239–249, <http://dx.doi.org/10.1145/1031495.1031524>.
- [27] C. Buragohain, S. Suri, Quantiles on streams, in: L. Liu, M.T. Özsu (Eds.), *Encyclopedia of Database Systems*, Springer US, Boston, MA, 2009, pp. 2235–2240, http://dx.doi.org/10.1007/978-0-387-39940-9_290.
- [28] C. Masson, J.E. Rim, H.K. Lee, DdsSketch: A fast and fully-mergeable quantile sketch with relative-error guarantees, *Proc. VLDB Endow.* 12 (12) (2019) 2195–2205, <http://dx.doi.org/10.14778/3352063.3352135>.
- [29] G.E. Pibiri, R. Venturini, Practical trade-offs for the prefix-sum problem, *Softw. - Pract. Exp.* 51 (5) (2021) 921–949, <http://dx.doi.org/10.1002/spe.2918>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.2918>.
- [30] P.M. Fenwick, A new data structure for cumulative frequency tables, *Softw. - Pract. Exp.* 24 (3) (1994) 327–336, <http://dx.doi.org/10.1002/spe.4380240306>, <https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.4380240306>.

- [31] A. Laaksonen, Guide to Competitive Programming - Learning and Improving Algorithms Through Contests, in: Undergraduate Topics in Computer Science, Springer, Cham, Switzerland, 2017, <http://dx.doi.org/10.1007/978-3-319-72547-5>.
- [32] S. Halim, Competitive Programming 4 : The Lower Bound of Programming Contests in the 2020s, Lulu.com, Singapore, 2020.
- [33] A. Liu, J. Lu, Y. Song, J. Xuan, G. Zhang, Concept drift detection delay index, IEEE Trans. Knowl. Data Eng. (2022) 1, <http://dx.doi.org/10.1109/TKDE.2022.3153349>.
- [34] N. Gans, G. Koole, A. Mandelbaum, Telephone call centers: Tutorial, review, and research prospects, Manuf. Serv. Opera. Manag. 5 (2) (2003) 79–141, <http://dx.doi.org/10.1287/msom.5.2.79.16071>, URL <https://pubsonline.informs.org/doi/abs/10.1287/msom.5.2.79.16071>.
- [35] L. Brown, N. Gans, A. Mandelbaum, A. Sakov, H. Shen, S. Zeltyn, L. Zhao, Statistical analysis of a telephone call center: A queueing-science perspective, J. Amer. Statist. Assoc. 100 (469) (2005) 36–50, <http://dx.doi.org/10.1198/016214504000001808>, URL <http://www.tandfonline.com/doi/abs/10.1198/016214504000001808>.
- [36] R. Bellman, R. Kalaba, On adaptive control processes, IRE Trans. Autom. Control 4 (2) (1959) 1–9, <http://dx.doi.org/10.1109/TAC.1959.1104847>, URL <http://ieeexplore.ieee.org/document/1104847/>.
- [37] Wannesm/dtaidistance v2.0.0, Zenodo, 2020, <http://dx.doi.org/10.5281/ZENODO.3981067>, URL <https://zenodo.org/record/3981067>.



Douglas O. Cardoso holds a bachelor's degree in Computer Science (2008), Master and Doctor degree in Systems and Computer Engineering (2012 and 2017, respectively), all from the Federal University of Rio de Janeiro. Previously a professor at the Federal Center for Technological Education Celso Suckow da Fonseca (CEFET/RJ), he currently works at Smart Cities Research Center of Polytechnic Institute of Tomar. The research topics he is currently interested in are Data Stream Mining, Machine Learning, and Network Science.



Thalys D. Galeno is an undergraduate student of Computer Engineering at the Federal Center for Technological Education Celso Suckow da Fonseca (CEFET/RJ). His interests are: Machine Learning, Data Stream Mining, and Big data.